

micROS.BT: An Event-Driven Behavior Tree Framework for Swarm Robots

Yunlong Wu^{1,2}, Jinghua Li², Huadong Dai^{1,*}, Xiaodong Yi¹, Yanzen Wang^{1,2}, and Xuejun Yang^{1,*}

Abstract—In this paper, we propose micROS.BT, an event-driven behavior tree (BT) framework aiming at supporting swarm-robot coordination. Compared with other BT frameworks, micROS.BT implements the event-driven way under the multi-thread mode, which can effectively save computing resources. Moreover, in order to ensure swarm-robot coordination, we optimize the implementation of the traditional blackboard and propose the multi-mode blackboard, which supports inner-tree, inter-tree, and inter-robot data sharing. Furthermore, considering the limited modularity of a single tree, micROS.BT realizes a mechanism called hierarchical tree management which involves inter-tree notifying and waiting functionalities, while ensuring that each tree is independent and self-scheduled. The effectiveness of micROS.BT is verified by simulation and real-robot experiments for different system settings, showing that a substantial improvement is achieved in comparison with the traditional BT implementations.

I. INTRODUCTION

Behavior tree (BT) is a typical control architecture that is widely used in computer games and robotics to describe the behaviors of AI and robots [1]. Compared with other control architectures, BT has the advantages of modularity, hierarchy, reactivity, readability, and reusability. There are many researches exploring the cutting-edge technologies of BT, such as automatic generation [2], verification [3], framework implementation [4], etc. BT framework implementation is the basis for realizing various innovative technologies, including optimizing the driving mechanism [5], designing new node types [6], enhancing the blackboard efficiency [7], etc. In this paper, we aim at exploring the way to build a distributed BT framework for swarm robots. We hope that this research can provide a reference for the extension of BT to swarm robots.

In the literatures of BT framework implementations, one of the most important aspects is the way to drive BT iterations. The typical driving method is the time-driven way, where the tree is executed from the root node and traverses the entire tree at a fixed frequency. Each traverse is called a *tick*. This way is simple to implement and supports multithreading, but it consumes extra CPU resources. Compared with the time-driven way, the event-driven way is more efficient, where the tree is ticked only when some event occurs, otherwise, the tree is halted. However, according to the survey of existing literatures, the current event-driven ways lack multi-thread implementations and is mainly used in computer games.

In addition to the driving mechanism, another core concept of BT is the *blackboard*, which is the main communication

medium among tree nodes [8]. In swarm robots, blackboard is one of the most important mechanism that guarantees the data sharing among robots and keeps the scalability of the swarm. However, the traditional blackboard implementation only focuses on the communication within a single tree, and lacks the inter-tree and inter-robot implementations [4].

Finally, in swarm-robot applications, a single BT may not be able to clearly demonstrate the behavior of a robot, especially when executing a complex task. Meanwhile, in order to take full advantage of the modularity of BT, there will be multiple BTs in one robot, and these BTs should be managed efficiently and hierarchically. As far as we know, the current implementation of BT lacks an effective framework for hierarchical tree management.

In order to overcome the above challenges, we propose micROS.BT, a distributed BT framework for swarm robots. micROS.BT is developed on micROS which is a morphable, intelligent, and collective robot operating system [9]. The main contributions of this paper are summarized as follows.

- An event-driven BT implementation for multi-thread environment is proposed. Firstly, the definitions of the event and event vector are given. Then, the synchronization mechanism between the BT engine and the action nodes is designed. Furthermore, in order to ensure the correct ticking sequence, the repulsive nodes and a halt mechanism are carefully constructed.
- A multi-mode blackboard is designed, which can effectively achieve inner-tree, inter-tree, and inter-robot communications. Inter-tree communication involves an elastic memory that can be dynamically expanded according to capacity requirements. In inter-robot communication, the technique of active message is introduced to adapt to practical communication environment.
- A hierarchical tree management method is introduced, which implements the inter-tree waiting and notifying mechanisms based on the inter-tree blackboard, the active message, and the inter-thread synchronization.

II. BACKGROUND: BT AND ACTIVE MESSAGE

A. Behavior Tree

BT is a directed tree structure used for behavior switching. The structure of BT is composed of *control flow nodes* and *execution nodes*, as shown in Fig. 1. The node symbols used in this paper are referenced in [1]. Except for the root node, each node has only one parent node, and each control flow node has at least one child node. In BT theory, there are four types of control flow nodes and two types of execution nodes [10]. The control flow nodes are Sequence, Fallback, Parallel,

¹Artificial Intelligence Research Center (AIRC), National Innovation Institute of Defense Technology (NIIDT), Beijing 100071, China

²Tianjin Artificial Intelligence Innovation Center (TAIIC), Tianjin 300457, China

*Corresponding authors: hddai@vip.163.com, xjyang@nudt.edu.cn

and Decorator, and the execution nodes are Condition and Action. When the parent node ticks its child node, the child node will start execution. Every time the node is ticked, it will return a status, which is one of *Success* (*S*), *Failure* (*F*), and *Running* (*R*). *S* and *F* denote that whether the behavior is executed successfully. *R* represents that the node has not been finished yet.

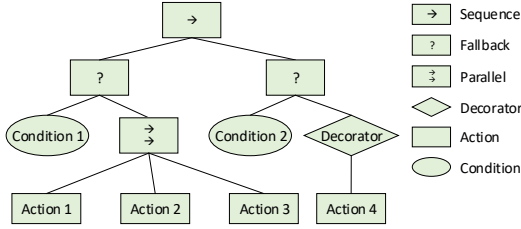


Fig. 1. An example BT structure.

Sequence: When the Sequence node receives the tick signal, it will pass the signal to its child nodes from left to right. When its child node returns *S*, the Sequence node will tick the next until the last node. If the currently ticked child node returns *F* or *R*, then it will return this status and stop ticking the subsequent child nodes.

Fallback: Fallback node starts ticking from the far left child node, and continues execution when the current child node returns *F*. If the current child node returns *S* or *R*, the Fallback node will return immediately and stop the operations of subsequent nodes.

Parallel: The Parallel node is used to perform multiple actions simultaneously. In the traditional BT implementation, the Parallel node has two parameters: *N* and *M*. *N* denotes the number of child nodes, and *M* indicates that when *M* child nodes return *S*, the Parallel node returns *S*. When *N* – *M* child nodes return *F*, the Parallel node returns *F*. In other cases, the Parallel node returns *R*.

Decorator: The Decorator node has only one child node and modify its tick process or return status.

Action: The Action node executes a specific behavior. In the single-thread BT implementation, the Action node runs in the main thread. While, in the multi-thread mode, the Action node can create additional threads to execute its behavior.

Condition: The Condition node is used to make judgments on the environment or the values on the blackboard. This node can only return *F* or *S*, not *R*, and it does not modify any environment variables or blackboard values.

B. Active Message

Active message is a communication mechanism used in parallel computing [11]. The key idea of active message is to embed instructions into the transmitted message, which can be inserted into the computation process of the receiver. Fig. 2 gives an implementation of active message, which mainly contains two parts: the user-mode active message API (Application Programming Interface) and the kernel-mode module. In user applications, the receiver often registers several handler functions to process the received messages, and the sender just calls the active message APIs to transmit

data. When a message is received by a specific robot, it is firstly obtained by the kernel module, then the message will be transmitted to the filter in the user mode. In the filter, the message is analyzed and then passed to the corresponding user-defined handler which can flexibly insert the message into the ongoing computation flow. In this paper, we adopt this active message implementation by default.

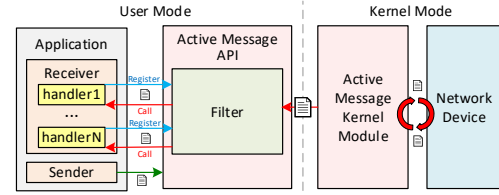


Fig. 2. An active message implementation.

III. THE PROPOSED FRAMEWORK

A. Framework Overview

The general diagram of micROS.BT is shown in Fig. 3, which contains two main parts: event-driven BT engine and multi-mode blackboard. The BT engine is responsible for correctly driving the tree according to the predefined events. All events are stored in an event vector in chronological order, and the BT engine is driven by the event in this vector. If there is no events in the event vector, the BT engine will be halted and then CPU resources are released. In our framework, a new event is generated when an Action node updates its status from *R* to *S* (or *F*), or sets values on the blackboard. For supporting swarm-robot applications, we have designed the multi-mode blackboard which is used to exchange data among nodes, trees and robots. The inner-tree blackboard can be only used to share data between the nodes in the same tree. If it is essential to transfer data among different trees, the inter-tree blackboard can be used. Furthermore, the inter-robot blackboard is designed to support communications among robots. All the three kinds of blackboards have formed the multi-mode blackboard.

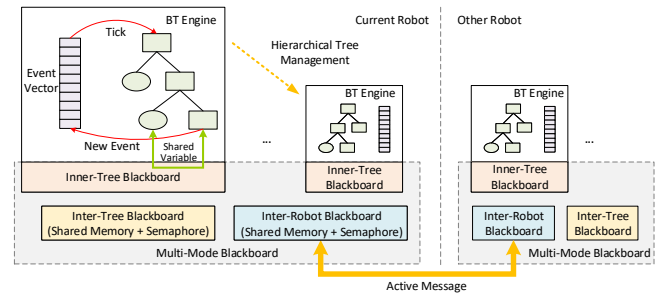


Fig. 3. General diagram of micROS.BT.

B. Event-Driven BT Engine

The BT engine is a controller that drives the tree iterations, which includes the event-driven way and the halt mechanism.

1) Event and Event Vector: We firstly define the *event* and its related data structure. When an Action node changes its status from *R* to *S* (or *F*) or updates a value on some blackboard, a new event is regarded to occur. For the Action

node i , its event is defined as (i, n_i) , where n_i represents the event number of the Action node i . n_i is initialized with 0, and at each time the event of this node is generated, it will increase by 1. Furthermore, the *event vector* of the tree is defined as $\mathbf{v}_{ev} = [(i, n_i), (j, n_j), \dots, (k, n_k)]$, where i, j, k generally denote the Action node IDs. A new event will be inserted into the back of $\mathbf{v}_{ev}$, while the BT engine processes the event from the beginning of $\mathbf{v}_{ev}$.

2) *Synchronization Between BT Engine and Its Action Nodes*: In our framework, the main BT engine control flow and Action nodes are in different threads. This implementation can improve the execution efficiency of Parallel nodes. Therefore, in order to realize the event-driven mechanism, the synchronization of the BT engine thread and the Action node threads becomes very important. With the event vector structure, a synchronization mechanism is designed, whose key idea is shown in Fig. 4. A C-like language is used to describe the entire event-driven process. As shown in Fig. 4(a), there are three Action nodes *Action 1*, *Action 2*, and *Action 3*. They call *push_back()* to insert their events $\{e_1, e_2, e_3\}$ into the event vector according to their generation time. The number in front of the C-like function indicates the time sequence. The front event of the event vector will be processed and allow *Action 1* to notify the BT engine. Then, the engine will be ticked from the root node. After performing a tick process, the front of the event vector will be erased and notify the waiting nodes, as shown in Fig. 4(b). The same process will be repeated unless the root node returns the terminal statuses (S or F) or the event vector is empty, as shown in Fig. 4(c) and Fig. 4(d).

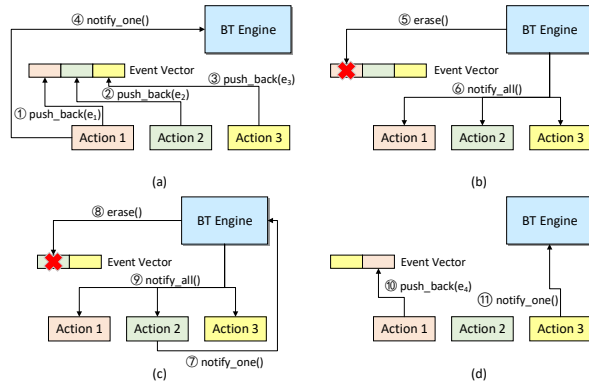


Fig. 4. An example to illustrate the synchronization mechanism between the BT engine and the Action nodes.

The main processes of the BT engine and the Action node are described in Alg. 1 and Alg. 2, respectively. In Alg. 1, the BT b is firstly created according to the input BT description file path p . Then, before the main loop of the engine, a unique lock κ_e with mutex τ_{em} is made, which will be used in synchronization between the BT engine thread and the Action node threads. In the main iterative process (line 3-line 9), the tree is firstly ticked and returns the status s of the root node. If s is the terminal status (i.e. S or F), the BT engine will be stopped by breaking the while loop. Otherwise, the engine thread will wait for the signal from the awakened Action node threads and then temporarily unlock

Algorithm 1: Main loop of the BT engine thread

Input: BT file path p , engine mutex τ_{em} , event vector $\mathbf{v}_{ev}$, engine condition variable γ_{ecv} , Action node condition variable γ_{acv}

Output: None

```

1  $b = createBT(p);$  // Build BT  $b$ 
2 Make a unique lock  $\kappa_e(\tau_{em});$ 
3 while not stopped do
4    $s = b.tick();$  // Traverse the tree
5   if  $s == S || F$  then
6     Break;
7    $\gamma_{ecv}.wait(\kappa_e);$  // Wait events
8   Erase the front of  $\mathbf{v}_{ev}$ ; // Delete  $\mathbf{v}_{ev}.begin()$ 
9    $\gamma_{acv}.notify\_all();$  // Awaken Action nodes

```

Algorithm 2: Event processing in the Action node thread

Input: Node ID i , event number n_i , engine mutex τ_{em} , Action node mutex τ_{am} , event vector $\mathbf{v}_{ev}$, engine condition variable γ_{ecv} , Action node condition variable γ_{acv} , time intervals $\Delta t_1, \Delta t_2$

Output: Whether the event is correctly processed

```

1 Insert  $(i, n_i)$  into the back of  $\mathbf{v}_{ev}$ ;
2 Make a unique lock  $\kappa_{ac}(\tau_{am});$ 
3 while  $\mathbf{v}_{ev}.begin() \neq (i, n_i)$  do
4   if  $isHalt()$  then
5     Erase the event  $(i, n_i)$  from  $\mathbf{v}_{ev}$ ;
6     Return false;
7    $\gamma_{acv}.wait\_for(\kappa_{ac}, \Delta t_1);$ 
8  $\kappa_{ac}.unlock();$ 
9 Make a unique lock  $\kappa_e(\tau_{em}, defer\_lock);$ 
10 while  $!\kappa_e.try\_lock()$  do
11   if  $isHalt()$  then
12     Erase the event  $(i, n_i)$  from  $\mathbf{v}_{ev}$ ;
13     Return false;
14    $sleep(\Delta t_2);$ 
15  $n_i = n_i + 1;$ 
16 // Set status or blackboard values
17  $\kappa_e.unlock();$ 
18  $\gamma_{ecv}.notify\_one();$  // Awaken the engine
19 Return true;

```

κ_e (line 7). When receiving the notify signal, the engine will erase the front event of the event vector $\mathbf{v}_{ev}$ and notify all the waiting Action node threads. Normally, the next waiting Action node thread will be triggered. τ_{em} , γ_{ecv} , and γ_{acv} are shared between the BT engine and the Action node threads.

Alg. 2 presents the process of event handling in the Action node thread. When the Action node changes its status from R to S (or F) or updates the blackboard values, this algorithm will be called. Firstly, a new event (i, n_i) is created and inserted into the back of $\mathbf{v}_{ev}$. Then, we will make a unique lock $\kappa_{ac}(\tau_{am})$, where τ_{am} is shared among Action node threads. Line 3-line 7 will check whether the current event is the next event to be processed. If it is, the execution continues, otherwise it will wait for the right event to be processed by the engine thread. In line 9-line 14, the Action node thread will try to lock the BT engine thread, and will obtain the lock when the engine is waiting (line 7 in Alg. 1).

Afterwards, the event number n_i will be increase by 1, and the Action node thread can execute its functionalities, e.g. setting node status or blackboard values. Finally, the Action node thread will wake the BT engine thread by notifying it.

3) *Halt Repulsive Action Nodes*: In order to ensure the correct ticking sequence, it is necessary to keep all the subsequent Action nodes halted before ticking the current Action node. For achieving this objective, we construct the basic data structure: *repulsive matrix* and *repulsive vector*. Alg. 3 presents the details of building the two structures. It firstly initializes repulsive matrix M and ancestor map $\mathbf{m}_{\text{anc}}$ for all Action nodes with zero. Then, it traverses each Action node $a \in \mathcal{A}$ in the tree, where $\mathcal{A}$ denotes the Action node set. Line 2-line 8 present the creation process of $\mathbf{m}_{\text{anc}}$, where $a.\text{getParent}()$ returns the parent node of Action node a . With the data structure $\mathbf{m}_{\text{anc}}$, in line 9-line 11, the repulsive matrix M is constructed. Finally, with the repulsive matrix, the repulsive vector $\mathbf{v}_{i,\text{rn}}$ is built for each Action node i .

Algorithm 3: Build repulsive matrix and repulsive vector

Input: Action node set $\mathcal{A}$
Output: Repulsive node vector $\mathbf{v}_{i,\text{rn}}$ for each Action node i

```

1 Initialize  $M$  and  $\mathbf{m}_{\text{anc}}$  with 0;
2 for each  $a \in \mathcal{A}$  do
3   Initialize  $\mathbf{v}_{\text{anc}}$  with 0;
4    $a_{\text{par}} = a.\text{getParent}()$ ;
5   while  $a_{\text{par}}$  exists do
6     Insert  $a_{\text{par}}$  into the front of  $\mathbf{v}_{\text{anc}}$ ;
7      $a_{\text{par}} = a_{\text{par}}.\text{getParent}()$ ;
8   Insert  $\{a, \mathbf{v}_{\text{anc}}\}$  into  $\mathbf{m}_{\text{anc}}$ ;
9 for each  $i \in [0, \mathcal{A}.\text{size}() - 1]$  do
10  for each  $j \in [i + 1, \mathcal{A}.\text{size}() - 1]$  do
11     $M_{i,j} = M_{j,i} = \text{areRepulsive}(a_i, a_j, \mathbf{m}_{\text{anc}})$ ;
12 for each  $i \in [0, \mathcal{A}.\text{size}() - 1]$  do
13  Initialize  $\mathbf{v}_{\text{tmp}}$  with 0;
14  for each  $j \in [0, \mathcal{A}.\text{size}() - 1]$  do
15    if  $M_{i,j} = 1$  then
16      Insert  $a_j$  into the back of  $\mathbf{v}_{\text{tmp}}$ ;
17   $\mathbf{v}_{i,\text{rn}} = \mathbf{v}_{\text{tmp}}$ ;
18 Return  $\mathbf{v}_{i,\text{rn}}, \forall i$ ;
```

Alg. 4 presents the details of judging whether two Action nodes are repulsive nodes. Firstly, the parent node IDs of a_i and a_j are obtained, which are stored in $k_{i,\text{par}}$ and $k_{j,\text{par}}$, respectively. If $k_{i,\text{par}}$ and $k_{j,\text{par}}$ are equal and are not Parallel nodes, then a_i and a_j are identified as repulsive nodes. Otherwise, the ancestor nodes of a_i and a_j will be checked. The key idea is firstly extracting the minimum value of the size between $\mathbf{m}_{i,\text{anc}}$ and $\mathbf{m}_{j,\text{anc}}$ which are ancestor vector of a_i and a_j , respectively. $\mathbf{m}_{i,\text{anc}}.\text{getNode}(d)$ returns the d -th item of $\mathbf{m}_{i,\text{anc}}$. Then, the ancestor nodes of a_i and a_j are compared from the root node. If the ancestor nodes of a_i and a_j are different, it means that they have different ancestor nodes here. If it is further determined that the last common ancestor node is not a Parallel node, then these two nodes are repulsive, otherwise they are non-repulsive nodes.

Algorithm 4: areRepulsive($a_i, a_j, \mathbf{m}_{\text{anc}}$)

Input: Action node a_i , Action node a_j , ancestor map $\mathbf{m}_{\text{anc}}$
Output: Whether a_i and a_j are repulsive nodes

```

1  $k_{i,\text{par}} = \mathbf{m}_{i,\text{anc}}.\text{back}().\text{getNodeID}()$ ;
2  $k_{j,\text{par}} = \mathbf{m}_{j,\text{anc}}.\text{back}().\text{getNodeID}()$ ;
3 if  $k_{i,\text{par}} == k_{j,\text{par}}$  then
4   Return  $a_i.\text{getParent}() \neq \text{Parallel}$ ;
5  $d = 0$ ;
6  $\varepsilon_{\text{min}} = \min(\mathbf{m}_{i,\text{anc}}.\text{size}(), \mathbf{m}_{j,\text{anc}}.\text{size}())$ ;
7 Create temporary node  $a_{\text{tmp}}$ ;
8 while  $d < \varepsilon_{\text{min}}$  do
9    $a_{\text{tmp}} = \mathbf{m}_{i,\text{anc}}.\text{getNode}(d)$ ;
10  if  $\mathbf{m}_{i,\text{anc}}.\text{getNode}(d) \neq \mathbf{m}_{j,\text{anc}}.\text{getNode}(d)$  then
11     $a_{\text{tmp}} = \mathbf{m}_{i,\text{anc}}.\text{getNode}(d - 1)$ ;
12    Break;
13   $d = d + 1$ ;
14 Return  $a_{\text{tmp}} \neq \text{Parallel}$ ;
```

With the repulsive matrix and repulsive nodes, the halt mechanism in control flow nodes can be achieved. When ticking an Action node, the control flow node firstly halts its repulsive nodes. The halt process is blocked and only returns when the repulsive nodes are indeed suspended.

C. Communication Mechanism: Multi-Mode Blackboard

In BT theory, blackboard is used to share data among the nodes in the same tree. In the traditional blackboard implementation, data is stored in the form of key-value pairs. In this paper, *setValue* and *getValue* are used as the writing and reading interfaces of the blackboard. For example, *setValue*(k, v, B) means to write key k and value v on blackboard B . In order to support swarm-robot applications, it is necessary to extend the blackboard to inter-tree and inter-robot implementations. Therefore, the multi-mode blackboard is proposed in this paper, including inner-tree, inter-tree, and inter-robot blackboards.

1) *Inner-Tree Blackboard*: The inner-tree blackboard is similar to the classic blackboard and mainly used for data exchange among nodes in a single tree. Considering the implementation of BT is mostly concentrated in one process with multiple threads, the inner-tree blackboard is designed as the shared variable on the heap. Each node in the same tree can access the data on the inner-tree blackboard through a pointer to it. For example, the pair (k, v) is a shared variable on the heap, and all the execution nodes in the same process can visit the inner-tree blackboard by calling *setValue*($k, v, B_{\text{inner-tree}}$) and *getValue*($k, v, B_{\text{inner-tree}}$).

2) *Inter-Tree Blackboard*: The inter-tree blackboard is used to share data among different trees in the same robot. The implementation of the inter-tree blackboard is mainly based on shared memory, and its mutually exclusive access depends on the semaphore mechanism, as shown in Fig. 5. In order to save memory resources as much as possible, an elastic memory design is adopted, in which the size of the inter-tree blackboard can be dynamically expanded.

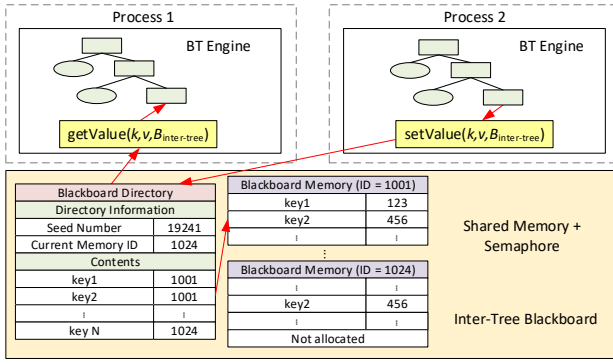


Fig. 5. *setValue* and *getValue* processes on the inter-tree blackboard.

For supporting this design, two tables are necessary to be built primarily: *blackboard directory* and *blackboard memory*, as shown in Fig. 5. The blackboard directory contains two main parts, one stores the directory information, and the other holds the keys and their corresponding blackboard memory IDs. In the directory information part, the seed number is a sign used to create a new blackboard memory. Each time a new memory is created, the seed number will increase by 1. The current memory ID keeps the newest blackboard memory ID. The blackboard memory is a fixed-size shared memory. When the current blackboard memory is not able to store a new key value, it will apply for a new blackboard memory. This process is transparent to users.

Algorithm 5: *setValue(k, v, b)*

Input: Key k , value v , inter-tree blackboard b
Output: Whether the value of k is correctly set

```

1  $id = checkKey(k);$ 
2 if  $id \geq 0$  then
3    $r = updateValue(k, v, id);$ 
4   if  $r == true$  then
5     Return  $true;$ 
6   else
7      $deleteKey(k, b);$ 
8      $m = getCurrentMemID(b);$ 
9      $t = updateValue(k, v, m);$ 
10    if  $t == true$  then
11       $addKeyToDirectory(k, b);$ 
12      Return  $true;$ 
13 else
14   // Do as line 8-12
15  $s = findSeed(b) + 1;$ 
16  $e = expandBlackboard(s, b);$ 
17 if  $e == false$  then
18   Return  $false;$ 
19  $updateValue(k, v, getCurrentMemID(b));$ 
20  $addKeyToDirectory(k, b);$ 
21 Return  $true;$ 

```

Alg. 5 introduces the process of setting inter-tree blackboard value. Firstly, we need to check whether key k exists in the blackboard directory (line 1). If it exists, we may modify k and its corresponding value (line 2-line 12), otherwise create a new key (line 13-line 18). When k exists,

Algorithm 6: *getValue(k, v, b)*

Input: Key k , returned value v , inter-tree blackboard b
Output: Whether the value of k is correctly obtained

```

1  $id = checkKey(k);$ 
2 if  $id \geq 0$  then
3    $r = findValue(k, v, id);$ 
4   if  $r == false$  then
5     Return  $false;$ 
6   Return  $true;$ 
7 else
8   Return  $false;$ 

```

its corresponding value will be updated (line 3). If the value is updated successfully, this function returns success. Otherwise, it means that the memory space occupied by the updated value exceeds the remaining space of the blackboard memory where k is located. Then, k should be removed from the directory (line 7), and *getCurrentMemID()* will be called to obtain the current blackboard memory ID. Afterwards, the key and its corresponding value will try to be added to the blackboard memory (line 9). If the addition is successful, update k in the directory and its corresponding memory ID. Line 13-line 18 present the case where k does not exist, and the process is the same as line 8-line 12. If the current blackboard memory cannot store the current value, a new blackboard memory needs to be applied. On this occasion, the seed needs to increase by 1, and expand a new blackboard memory according to the seed (line 15-line 16). The value v will be stored in the current memory. Here it is assumed that the size of each value will not exceed the capacity of a blackboard memory. Alg. 6 gives the process of getting a value from the blackboard. Compared with the *setValue* process, *getValue* only needs to query the value corresponding to the given k (line 3). If k exists, return its value and return *true*, otherwise return *false*.

3) *Inter-Robot Blackboard*: In order to support data exchange among robots, the inter-robot blackboard is proposed, which is implemented in a distributed manner, that is, in each robot, there will be a shared memory to store part of the inter-robot blackboard. When a robot intends to send data to another robot, it calls the API *setValue(k, v, d, B_{inter-robot})*, where d is the receiver robot ID, and $B_{inter-robot}$ represents the pointer to the inter-robot blackboard. When the user calls the *setValue* interface, the k and v will be packaged, and d will be inserted into the header. After investigating existing works, we think that active message is very suitable as the underlying communication mechanism of inter-robot blackboard [11]. The encapsulated data packet will be sent to the receiver robot via active message. In the receiver robot, the received data packet will trigger the *handler*, which has been registered in active message. The functionalities of *handler* include two parts: writing the received key and value to the local shared memory, and invoking the callback functions registered by the nodes of BT, as shown in Fig. 6. In the user-defined callback functions, a typical work is to call the *getValue* interface function to obtain the key and value in the local shared memory.

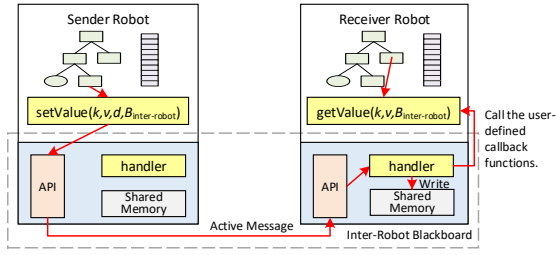


Fig. 6. An example to illustrate the key idea of the inter-robot blackboard.

D. Hierarchical Tree Management

Adopting multiple BTs to describe a task has the advantages of higher modularity and better maintainability than using a single BT. The key to implement multiple BT management is to design a reasonable synchronization mechanism to ensure the correctness of multiple BT scheduling. In this section, an active notifying mechanism is proposed based on active message, which can be seen as a kind of condition variable implementations in multi-process environment.

1) *Periodically Checking*: In the implementation of micROS.BT, communication among trees in the same robot depends on the inter-tree blackboard. Therefore, an intuitive way to achieve inter-tree management is to make a tree periodically check a certain key-value pair on the inter-tree blackboard written by other trees. For example, some execution node in the high-level tree continues to check an agreed key-value pair that will be written by the execution node in the low-level tree. When the execution node in the low-level tree completes its task, it sets the value of the agreed key, and the node in the high-level tree will get the value and perform the corresponding control flow. However, it is not an efficient inter-tree management method, because the high-level tree needs to constantly check the values on the inter-tree blackboard and wastes CPU resources. Therefore, a better solution is that, initially the node of the high-level tree waits for the keys and releases CPU resources. When the low-level tree writes the agreed value on the inter-tree blackboard, the high-level tree will be notified and continue execution.

2) *Actively Notifying*: With active message, a hierarchical tree control is implemented with active notifying. Three inter-tree interfaces have been designed: *notifyOne*, *notifyAll* and *wait*. *notifyOne*($k, v, B_{\text{inter-tree}}$) is used to wake up one node in other trees, which calls *wait* to wait for the same key k , as shown in Fig. 7(a). When a node calls *wait*($k, v, B_{\text{inter-tree}}$), the corresponding thread will be blocked until some node calls *notifyOne* or *notifyAll*. Similarly, *notifyAll*($k, v, B_{\text{inter-tree}}$) can wake up all nodes waiting for the key k , as shown in Fig. 7(b).

The specific implementation is that when a node calls *notifyOne* or *notifyAll*, the key k and value v are packaged and sent to the active message kernel module. In the receiver, the received message will trigger the user-registered *handler* which mainly accomplishes two things: writing the key-value pair to the shared memory of the inter-tree blackboard and using the condition variable to wake up one or all nodes that are waiting for the key.

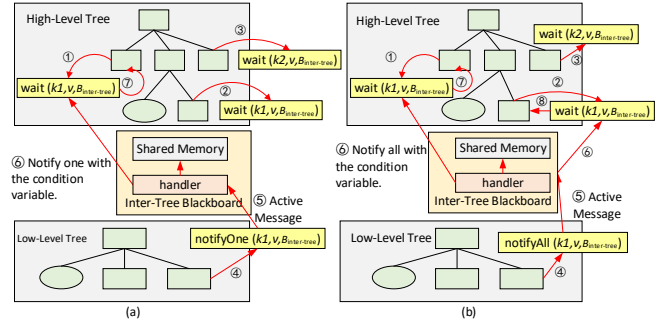


Fig. 7. *notifyOne*, *notifyAll* and *wait* mechanisms.

E. A BT Design Pattern For Swarm Robots

Considering the scalability and design complexity, it is unrealistic and inefficient to design a tree for each robot individually. Therefore, an effective BT design pattern for swarm robots is needed. We refer to the idea of SPMD (Single Program Multiple Data) in parallel computing, which recommends the way that the same program runs on different processes, and each process executes its branch according to different data inputs [12]. Motivated by its idea, we propose that the tree on each robot has the same structure, while each node is data-driven, that is, it displays different behaviors according to different data inputs. Similar to SPMD, this design pattern is called *Single Tree Multiple Robots* (STMR).

IV. EXPERIMENTS

A. CPU Utility Comparison

The CPU usage is an important indicator to measure the performance of a BT engine. In this test, we compare the CPU usage of the event-driven way and the time-driven way for running a single-level BT with different ticking periods. The BT structure is shown in Fig. 8(a), *Condition 1* initially returns F and *Action 1* will create a new thread to execute its task. In order to accurately measure the ticking cost, the thread of *Action 1* only performs sleeping operations and releases CPU resources. We select six kinds of on-board computers to run the tests, including ODROID-XU4, NVIDIA Jetson TX2, UP Board, Raspberry Pi, Intel i7, and NVIDIA Jetson Xavier NX. The results are shown in Fig. 9, which can be seen that the performance of the event-driven way exceeds most of the time-driven ways.

In addition, one advantage of micROS.BT lies in the hierarchical tree management. In order to test its performance, a two-level BT is conducted, as shown in Fig. 8(b). As the test above, the threads of the Action nodes only perform sleeping operations, and the results are shown in Fig. 10. In the middle of the timeline, the low-level BT execution is completed, thus the CPU usage decreases significantly. As can be seen from the figure, event-driven way can effectively reduce CPU usage compared to time-driven way as the single-level BT testing. Moreover, for the time-driven way, as the levels of BTs increase, the CPU usage also rises proportionally, which restricts its scalability. However, for the event-driven way, the CPU usages in the single-level BT and the two-level BT are almost the same. Therefore, it can be concluded that the

event-driven way in micROS.BT has more scalability than the traditional time-driven way.

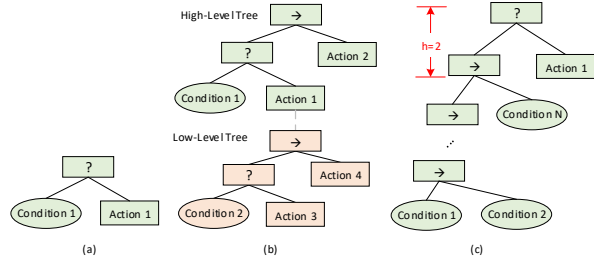


Fig. 8. CPU usage testings for (a) single-level tree, (b) two-level tree, and (c) different heights.

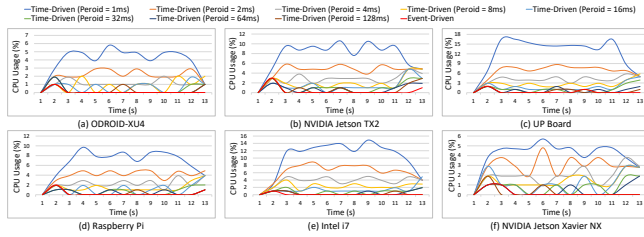


Fig. 9. CPU usage comparison for the single-level BT testing.

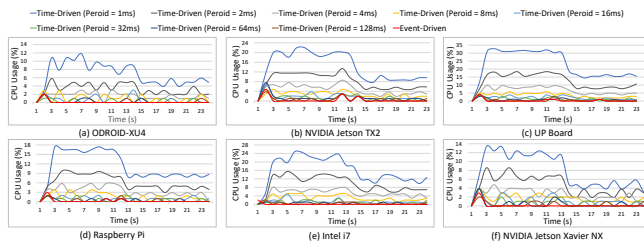


Fig. 10. CPU usage comparison for the two-level BT testing.

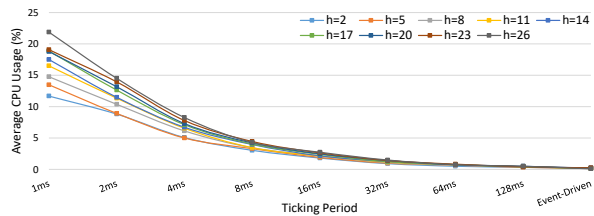


Fig. 11. Average CPU usage comparison between the event-driven way and the time-driven ways with different ticking periods.

Besides the ticking period, the CPU usage is also related to the number of traversed tree nodes. The more tree nodes traversed in each ticking process, the higher the CPU usage. We define the height h of a BT as shown in Fig. 8(c). The tree has one Action node and several Condition nodes, and the ellipsis denotes the same branch structure is repeated. We test the average CPU usage in different heights, the results are shown in Fig. 11. It is shown that the CPU usage for the time-driven way increases significantly with the ticking period decreasing. On the contrary, the CPU usage of the event-driven way is hardly unchanged, which is suitable for large-scale BT building.

B. Elastic Memory Scalability Performance

The inter-tree blackboard is used for data sharing among BT nodes in the same robot. In micROS.BT, the inter-tree

blackboard is composed of multiple fixed-size blackboard memories, and it can dynamically expand the storage capacity. In order to test the scalability of the inter-tree blackboard, we keep writing fixed-size key-value pairs (36 bytes) to the inter-tree blackboard and recording the occupied number of blackboard memories, the results are shown in Fig. 12. From the figure, it is indicated that as the size of input data grows, the more blackboard memories are allocated, which proves that compared to a fixed-size blackboard, elastic memory can utilize memory space more efficiently.

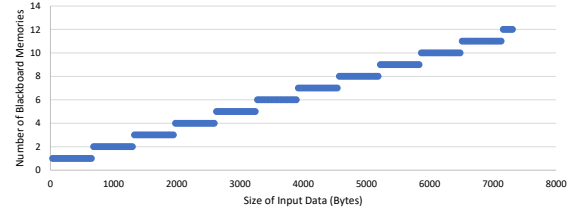


Fig. 12. Number of blackboard memories as the size of input data increases.

C. Swarm-Robot Surveillance Demo in Gazebo

In order to verify the usability of micROS.BT in swarm-robot applications, we build a system consisting of six fixed-wing unmanned aerial vehicles (UAVs) in Gazebo simulation environment. The UAV models are referenced from the open source project ROSplane of the MAGICC laboratory of the Brigham Young University (BYU) [13]. With these models, we consider a scenario where six UAVs continue to execute a surveillance task, which is described by the BTs in Fig. 13. The high-level tree is responsible for controlling the task execution flow. In this test, there are a total of two tasks: surveillance and return. Surveillance is the main task, but when the battery power is low, the UAV will be ordered to return home. Each task corresponds to a low-level BT, which is used for path planning and path following.

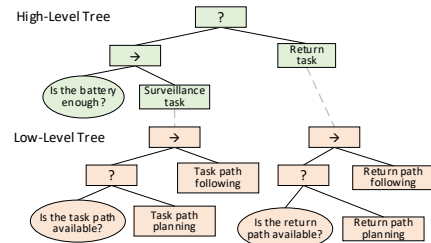


Fig. 13. BT structure of the swarm-robot surveillance demo.

We further simulate that the task execution process continues to consume battery power, and the *Task path following* node ceaselessly checks the power. When the battery is not enough, it will call the *notifyOne* interface to generate an event in the high-level tree and write the power information on the inter-tree blackboard. Then, the high-level tree will traverse the entire tree driven by the event and trigger the *Return task* node. For the surveillance task, the leader-follower control mode is adopted, which means that the leader will continuously send its own pose through the inter-robot blackboard, and the follower will continuously adjust its pose according to keep synchronization with the leader.

Fig. 14 demonstrates the whole trajectories of the six UAVs in a top view. The trajectories are divided into four segments: takeoff, fly to the planned path, execute surveillance task, and return. The first three segments belong to the surveillance task, and the last one belongs to the return task.

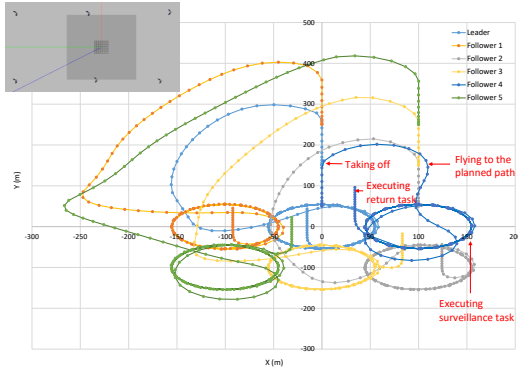


Fig. 14. The recorded trajectories of the six fixed-wing UAVs.

D. Swarm-Robot Demo with Crazyflie 2.1

We further explore to deploy micROS.BT in real robots. As an attempt, the Bitcraze Crazyflie 2.1 quadrotor is selected as the test platform, which carries an STM32F4 processor and 192kB random-access memory. Considering the performance limitation of STM32F4, we use three ODROID-XU4 platforms as computers, and connect each one to a quadrotor with a Crazyradio PA module, as shown in Fig. 15.



Fig. 15. Swarm-robot experiment with Bitcraze Crazyflie 2.1 quadrotors.

The same BT structure is adopted as shown in Fig. 13, while the implementation of the execution nodes is changed. Firstly, three quadrotors take off simultaneously and fly around their planned waypoints. After a period of time, the leader quadrotor sends a displacement command to each follower quadrotor through the inter-robot blackboard. Follower quadrotors move a specified distance and continue to fly around the new planned waypoints. Finally, when the battery is insufficient, the quadrotor executes the return task and land. The CPU usage is shown in Fig. 16, which demonstrates that the CPU usage of the three computers are all less than 10% and proves that the event-driven way of micROS.BT can effectively save CPU resources.

V. CONCLUSION

This paper designed and implemented an event-driven BT framework micROS.BT for swarm robots. The framework

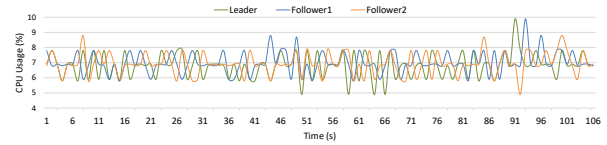


Fig. 16. The recorded CPU usages of the three ODROID-XU4 computers. adopted the event-driven way, which can effectively reduce CPU usage compared with the time-driven way. In order to support swarm-robot coordination, micROS.BT introduced the multi-mode blackboard, which had the capabilities of inner-tree, inter-tree, and inter-robot communications. Combined with the inter-tree blackboard and active message, we had achieved a hierarchical tree management mechanism that can effectively increase the readability of complex tasks. Finally, the simulation and real-robot experimental results showed that micROS.BT can not only support the swarm-robot applications, but also improve system performance.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China under Grant No. 61906212. The authors would like to thank Jiexin Zhang, Meng Xu, Qingwei Chen, Xiaoqing Zhao, Hao Li, and Zhenzhen He for their help in this work.

REFERENCES

- [1] M. Colledanchise and P. Ögren, "How behavior trees modularize hybrid control systems and generalize sequential behavior compositions, the subsumption architecture, and decision trees," *IEEE Trans. Robot.*, vol. 33, no. 2, pp. 372–389, 2016.
- [2] M. Colledanchise, R. M. Murray, and P. Ögren, "Synthesis of correct-by-construction behavior trees," in *Proc. IEEE/RISJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2017, pp. 6039–6046.
- [3] O. Biggar and M. Zamani, "A framework for formal verification of behavior trees with linear temporal logic," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2341–2348, 2020.
- [4] M. Colledanchise, A. Marzino, D. V. Dimarogonas, and P. Ögren, "The advantages of using behavior trees in multi-robot systems," in *Proc. Int. Symp. on Robotics (ISR)*, 2016, pp. 1–8.
- [5] E. Safranov, M. Vilzmann, D. Tsetserukou, and K. Kondak, "Asynchronous behavior trees with memory aimed at aerial vehicles with redundancy in flight controller," in *Proc. IEEE/RISJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2019, pp. 3113–3118.
- [6] B. Merrill, "Building utility decisions into your existing behavior tree," *Game AI Pro 360: Guide to Architecture*, pp. 81–90, 2019.
- [7] A. Shoulson, F. M. Garcia, M. Jones, R. Mead, and N. I. Badler, "Parameterizing behavior trees," in *Proc. Int. Conf. on Motion in Games (MIG)*, 2011, pp. 144–155.
- [8] D. Hu, Y. Gong, B. Hannaford, and E. J. Seibel, "Semi-autonomous simulated brain tumor ablation with ravenii surgical robot using behavior tree," in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2015, pp. 3868–3875.
- [9] X. Yang, H. Dai, X. Yi, Y. Wang, S. Yang, B. Zhang, Z. Wang, Y. Zhou, and X. Peng, "micros: a morphable, intelligent and collective robot operating system," *Robot. Biomimet.*, vol. 3, no. 1, pp. 1–9, 2016.
- [10] M. Colledanchise, A. Marzino, and P. Ögren, "Performance analysis of stochastic behavior trees," in *Proc. IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2014, pp. 3265–3272.
- [11] T. Von Eicken, D. E. Culler, S. C. Goldstein, and K. E. Schauer, "Active messages: a mechanism for integrated communication and computation," *ACM SIGARCH Computer Architecture News*, vol. 20, no. 2, pp. 256–266, 1992.
- [12] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.
- [13] G. Ellingson and T. McLain, "Rosplane: Fixed-wing autopilot for education and research," in *Proc. Int. Conf. on Unmanned Aircraft Systems (ICUAS)*, 2017, pp. 1503–1507.